Presence Monitoring — AI System

An mmWave radar-based system that monitors a senior resident's daily presence, classifies posture using a Random Forest model, and raises smart alerts for prolonged inactivity or unusual patterns.

Table of Contents

Overview

The Virtual Village Presence Monitoring System ingests raw mmWave radar JSON data, classifies a senior resident's posture every ~7 seconds using a trained Random Forest model, and raises smart alerts when unusual or potentially dangerous patterns are detected.

Core idea: The radar tracks the resident's vertical height (z1) as they move between rooms. Standing (~1.2m), sitting (~0.6m), and lying down (~0.2m) produce distinct z1 signatures the model learns to classify reliably.

End-to-end flow:

```
Raw radar JSON  →  EDA Notebook  →  Feature Engineering
                →  Random Forest Model  →  Posture Labels
                →  Alert Engine  →  Dashboard / Alerts
```

160,297	14 Days	6 Rooms	~7 sec	1 Person	3 Classes
Total Readings	Data Duration	Sensor Coverage	Sample Rate	Senior Resident	Posture Labels

Project Structure

File	Description
`Notebooks/01_FINAL EDA.ipynb`	Exploratory data analysis — daily patterns, room usage, posture zones
`Notebooks/02_Posture_Classification_And_Alerts.py`	Random Forest model training, evaluation, alert system, full visualization
`Notebooks/02_Posture_Model.py`	Lightweight posture classification + alert logic (no visualization)
`Notebooks/Random Forest model.ipynb`	Model training notebook with metrics and feature importance
`Notebooks/sensor_data.csv`	Preprocessed feature-ready CSV exported from EDA
`Notebooks/VV_Complete_System.png`	Complete system architecture / pipeline overview image
`Live Demos/dashboard_app.py`	Streamlit live monitoring dashboard
`Live Demos/alert_demo2.py`	Scenario demo: Living Room lying-down alert at 2pm
`Live Demos/sensor_data.csv`	Copy of processed CSV used by the dashboard and demo
`Data/presence_data_4.json`	Raw mmWave radar JSON — 160,297 readings across 14 days
`requirements.txt`	Python dependencies — install with: pip install -r requirements.txt

Dataset

The raw data file presence_data_4.json contains 14 days of continuous mmWave radar readings collected across 6 rooms of a single-occupancy home. Each record includes:

- radarUuid — maps to a specific room sensor
- createTime — timestamp at YYYYMMDDHHmmSS format
- parsedData — 8 values: `x_pos, y_pos, z_height, x_motion, y_motion, z_motion, signal_strength, target_count`

Sensor-to-room mapping:

- S0001 → Living Room

- S0002 → Bedroom
- S0003 → Kitchen
- S0004 → Bathroom
- S0005 → Study
- S0006 → Hallway

Note: target_count is always 1 (single resident). x_motion, y_motion, z_motion were dropped due to near-perfect correlation with positional features (0.90–0.99).

Quickstart

1. Install Dependencies

From any terminal with Python available:

```
pip install -r requirements.txt
```

2. Update the Data Path

Every script and notebook contains a FILE variable near the top. Update it to point to your local copy of presence_data_4.json:

```
FILE = r'C:\Your\Path\To\Data\presence_data_4.json'
```

On Mac or Linux:

```
FILE = '/your/path/to/Data/presence_data_4.json'
```

3. Run in Order

1. Open and run 01_FINAL EDA.ipynb in Jupyter
2. Run 02_Posture_Classification_And_Alerts.py for the full model + alert pipeline
3. Run 02_Posture_Model.py for a lightweight model-only version
4. Launch the dashboard: streamlit run "Live Demos/dashboard_app.py"
5. Run the alert demo: python "Live Demos/alert_demo2.py"

Pipeline Walkthrough

EDA Notebook (01_FINAL EDA.ipynb)

Performs full exploratory analysis of the raw JSON data and produces the main EDA dashboard image.

6. Loads and parses presence_data_4.json into a structured DataFrame

7. Computes per-sensor record counts and sampling interval statistics
8. Confirms single-resident tracking (target_count always = 1)
9. Defines posture zones from z_height distribution
10. Generates hourly and daily routine fingerprints
11. Produces a full 4-row visual dashboard saved as VirtualVillage_EDA_Dashboard.png

Key outputs:

- Daily Routine Fingerprint — average z_height by hour with labeled activity phases
- Posture Distribution — Standing (71.4%), Lying Down (32.8%), Sitting, Unknown
- 14-Day Heatmap — room-by-hour activity with green/red color coding
- Feature Correlation Matrix — revealing redundant motion features for removal

Posture Classification Model (02_Posture_Classification_And_Alerts.py)

Trains and evaluates a Random Forest classifier on posture labels derived from z_height thresholds.

Features used:

- x_pos, y_pos — horizontal position
- z_height — vertical height (primary posture indicator)
- signal_strength — radar signal quality

Target labels:

- Lying Down → z_height 0.05 – 0.40 m
- Sitting → z_height 0.41 – 0.80 m
- Standing → z_height 0.81 – 1.75 m
- Unknown → outside all zones

Training pipeline:

12. Labels are derived from z_height thresholds (no manual annotation needed)
13. 80/20 train/test split with stratification
14. 5-fold cross-validation for robustness
15. Classification report generated: precision, recall, F1 per class
16. Confusion matrix and feature importance chart produced

Smart Alert System

Two alert patterns run on top of the model's real-time posture output:

⚠ Alert Pattern 1 — Lying During Active Hours
Triggered when the resident is detected lying down between 8:00 AM and 8:00 PM. Flags potential daytime falls or unexpected resting.

⚠ Alert Pattern 2 — Prolonged Same Posture
Triggered when the same posture persists for 45+ minutes during active hours (8am–8pm). Flags inactivity that may indicate distress or a fall.

Alert records include timestamp, sensor/room, detected posture, duration, and alert type. Results are saved to a CSV for downstream review.

Lightweight Model Script (02_Posture_Model.py)

A stripped-down version of the classification pipeline — no visualization dependencies. Useful for testing the model logic or running in minimal environments.

17. Loads and parses presence_data_4.json
18. Trains Random Forest on posture labels
19. Runs alert logic on predictions
20. Prints results to terminal — no matplotlib or seaborn required

```
python 02_Posture_Model.py
```

Streamlit Dashboard (Live Demos/dashboard_app.py)

A live monitoring dashboard built with Streamlit. Displays real-time posture state, room occupancy, alert history, and daily routine summary.

To launch:

```
streamlit run "Live Demos/dashboard_app.py"
```

- Reads Live Demos/sensor_data.csv for processed readings
- Auto-refreshes to simulate a live sensor feed
- Requires Streamlit and Plotly — both included in requirements.txt

Demo Files (Live Demos/)

The Live Demos folder contains everything needed to run a standalone presentation — no Jupyter required.

alert_demo2.py

A scenario demo simulating a realistic alert sequence: normal morning activity followed by a lying-down event at 2:00 PM in the Living Room, triggering both Pattern 1 and Pattern 2.

```
python "Live Demos/alert_demo2.py"
```

dashboard_app.py + sensor_data.csv

The Streamlit dashboard and its required CSV are both included in this folder so the live demo runs self-contained without needing the full Notebooks directory.

Posture Zone Definitions

Zones are derived empirically from the z_height distribution of the actual sensor data — no manual calibration is required.

```
z_height 0.05 – 0.40 m  →  Lying Down   (sleeping / resting on floor or bed)
z_height 0.41 – 0.80 m  →  Sitting      (chair, sofa, toilet)
z_height 0.81 – 1.75 m  →  Standing     (walking, cooking, active)
Outside all ranges      →  Unknown      (sensor noise or out-of-range reading)
```

Mean z_height across all readings: 0.82m (consistent with a predominantly standing or sitting resident during daytime hours).

Notes & Assumptions

- Single resident — all models and alerts assume exactly one person tracked at a time (target_count = 1 in all records).
- Hardcoded file path — every script and notebook requires the FILE variable to be updated to the local path of presence_data_4.json before running.
- Labels are rule-based — posture labels are derived from z_height thresholds, not manually annotated ground truth. Model accuracy reflects how well RF reproduces these rules.
- No real-time sensor connection — the system replays historical data. For live deployment, a sensor API integration layer would be required.
- Sampling rate — ~7 second intervals per sensor. Each room produces an independent stream; the resident can only be in one room at a time.
- Active hours definition — alert patterns use 8:00 AM to 8:00 PM as the active window. This can be adjusted in the alert engine.

Future Improvements

- Real-time sensor integration — connect to live mmWave radar API for streaming predictions
- Caregiver mobile alerts — push notifications to family or nursing staff when alerts fire
- Fall detection module — extend to the dedicated XGBoost fall detection pipeline (see Fall_model_Documentation.pdf)
- Multi-resident support — generalize alert and tracking logic for households with more than one occupant
- Anomaly detection — flag deviations from the resident's established daily routine fingerprint

- Ground truth labels — collect manually annotated fall/non-fall events for supervised model retraining